# documentation (exercise 1)

Roberto Sonzini Gobbi: 12505287

October 2025

For the assignment I followed the honor code:

- Rule 1: Don't look at solutions or code of other students; everything you submit should be your own work
- Rule 2: Don't share your solution code with others; however, discussing ideas or general strategies is fine and encouraged
- Rule 3: If you use code from somewhere you need to cite the source directly in your code (e.g., StackOverflow). If the source is a generative AI tool (e.g., ChatGPT) provide a link to the specific Chat. I know that I have full responsibility for what I submit. I made sure the assignment allows the teaching staff to evaluate which competences I have acquired. I am aware that plagiarism is prohibited and may lead to failure of this course.

# 1 Experiment 1

## 1.1 Question 1

The kernel function itself is a Gaussian function. To apply it to a discrete image, we need to discretize the function and define the dimensions of the matrix that represents the kernel. To achieve this, we determine a kernel width (defined as: $2\lceil 3\dot{\sigma}\rceil + 1$). This width ensures that the kernel matrix retains the essential information of the Gaussian distribution, including both the peak and the tails, as shown in Figure 2.

Each value in the kernel corresponds to a weight derived from the Gaussian function. These weights determine how much each neighboring pixel contributes to the new value of the center pixel. Pixels closer to the center have higher weights, meaning they influence the filtered value more strongly, while pixels farther away have lower weights, contributing less. Increasing $\sigma$ spreads the Gaussian, giving relatively more weight to distant pixels and resulting in a stronger blur. This is important to achieve an effective blurring effect, where each new pixel value is computed as a weighted average of its neighboring pixels. The contribution of all the pixels within the kernel grid produces the characteristic Gaussian smoothing.

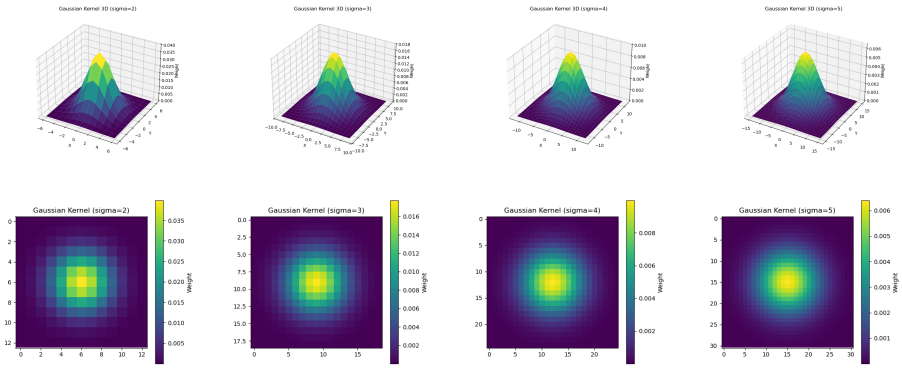
In practice, increasing $\sigma$ expands the region of influence, meaning that more neighboring pixels are included in the computation (as illustrated in Figure 3).

Choosing an inappropriate $\sigma$ can affect the subsequent steps of the algorithm. A large $\sigma$ results in a smoother gradient and may reduce the ability to detect edges or contours, while a very small $\sigma$ may fail to adequately suppress noise, leading to spurious edges. Finally, applying Gaussian blurring is a necessary preprocessing step to reduce image noise (as further discussed in Experiment 3).



(a) $\sigma = 2$ (b) $\sigma = 3$ (c) $\sigma = 4$ (d) $\sigma = 5$

Figure 1: Comparison of images for different $\sigma$ values.




(a) $\sigma = 2$ (b) $\sigma = 3$ (c) $\sigma = 4$ (d) $\sigma = 5$

Figure 2: Comparison of Kernel for different $\sigma$ values.

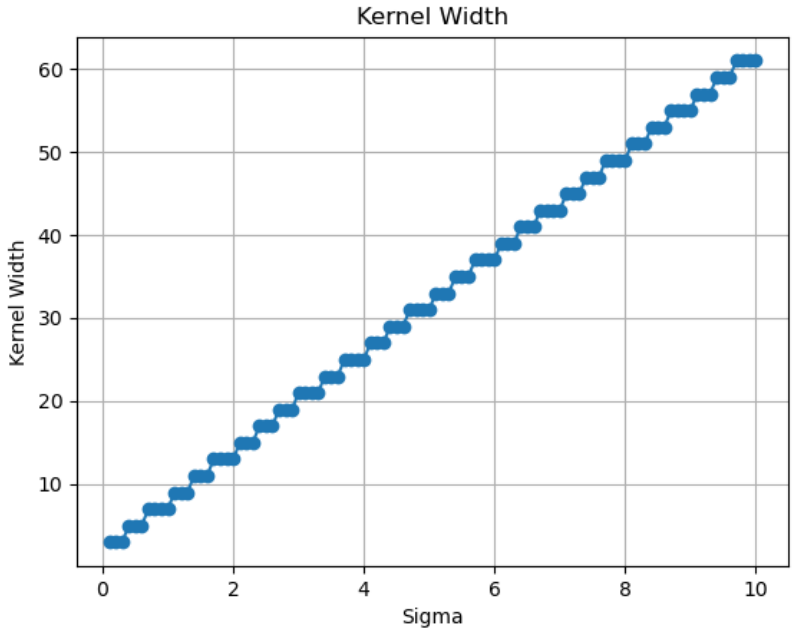
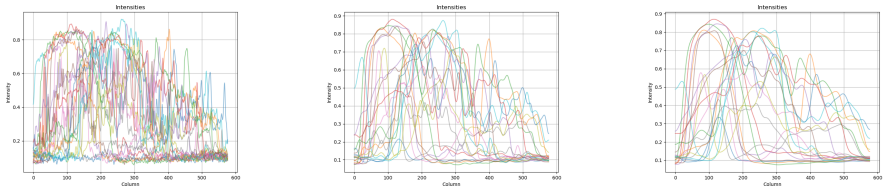



Figure 3: kernel width (# pixels) depending on $\sigma$ values.

### 1.2 Question 2

By applying the gaussian kernel we can observe that the row intensities become more and more smooth. The stronger the blur (i.e., the higher the value of $\sigma$), the smoother the intensities become. This is because the Gaussian filter effectively averages each pixel with its neighboring pixels, producing a smoothing effect.



(a) $\sigma = 1$ (b) $\sigma = 5$ (c) $\sigma = 10$

Figure 4: Comparison of row intensities for stronger blur.

## 2 Experiment 2

### 2.1 Question 1

Changing the threshold values has a strong impact on the results of the algorithm.

The low threshold determines which gradients are initially considered as potential edges, based on the difference between neighboring pixels. If this parameter is too low (Figure 5a), we may detect false edges. For example, a marked shadow may create strong differences between neighboring pixels even though it is not a real edge. On the other hand, setting the low threshold too high (Figure 5b) makes the algorithm overly conservative: only very strong gradients are considered, and some true edges may be missed.

The high threshold is used to identify the true edges. The idea is that a true edge must contain at least one gradient value higher than the high threshold, since edges correspond to significant differences between neighboring pixels. Setting the high threshold too high may cause real edges to be discarded, while setting it too low (Figure 5d) can lead to false positives, because weak gradients are not discarded.

This explanation is intuitive; in practice, finding the optimal threshold values can be challenging due to the variability of images. Factors such as lighting, shadows, and colors can affect the results. For instance, shadows can increase the likelihood of detecting false edges, and when an image contains both light and dark regions, it may be difficult to choose threshold values that correctly detect edges across the whole image.



(a) (b) (c) (d)

Figure 5: Comparison with different thresholds.

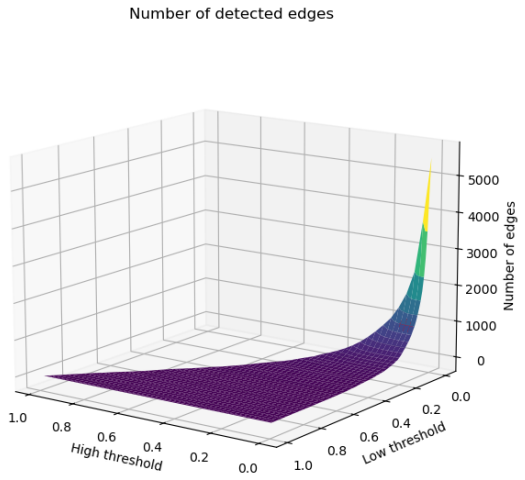



Figure 6: The plot illustrates the number of detected edges for different threshold values. It is evident that the choice of thresholds significantly affects the number of detected edges. The quantitative results shown in the plot support the qualitative analysis presented earlier.

## 2.2 Question 2

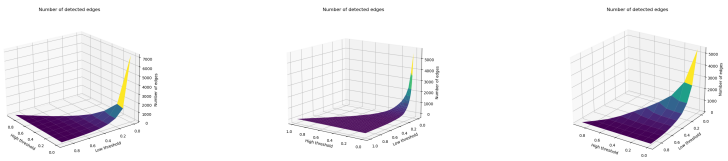
In principle keeping the same thresholds for different images can lead to have different behaviours of the algorithm. In fact we could have that with specific thresholds the canny algorithm will work well, while on another image it could work not that good. We could try to use the same thresholds when we have similar images in terms of gray scale so that the range of the gradient could oscillate in the same ranges for the two images.



(a) (b)

Figure 7: Comparison with on different images.

As we can see from the Figure 8 the thresholds used for the 7b image work a little worse on the 7a since we can see that we are finding a lot of useful borders. We can suppose that for the 7a image we have a too low high threshold for instance.



(a) beardman image (b) rubens image (c) parliament image

Figure 8: The plot compares the number of detected edges as a function of threshold values. As can be seen, different images produce distinct plots, which indicates that — as discussed above — the same thresholds cannot be applied to different images without affecting the quality of the results.

# 3 Experiment 3

## 3.1 Question 1

Adding Gaussian noise to the image makes the detected edges increasingly inaccurate. With a reasonably low noise level (i.e., low standard deviation), some true edges can still be detected, although the number of false positives increases. Other parameters, such as the Gaussian blur $\sigma$ and the thresholds, can be adjusted to achieve more acceptable results under low noise conditions.

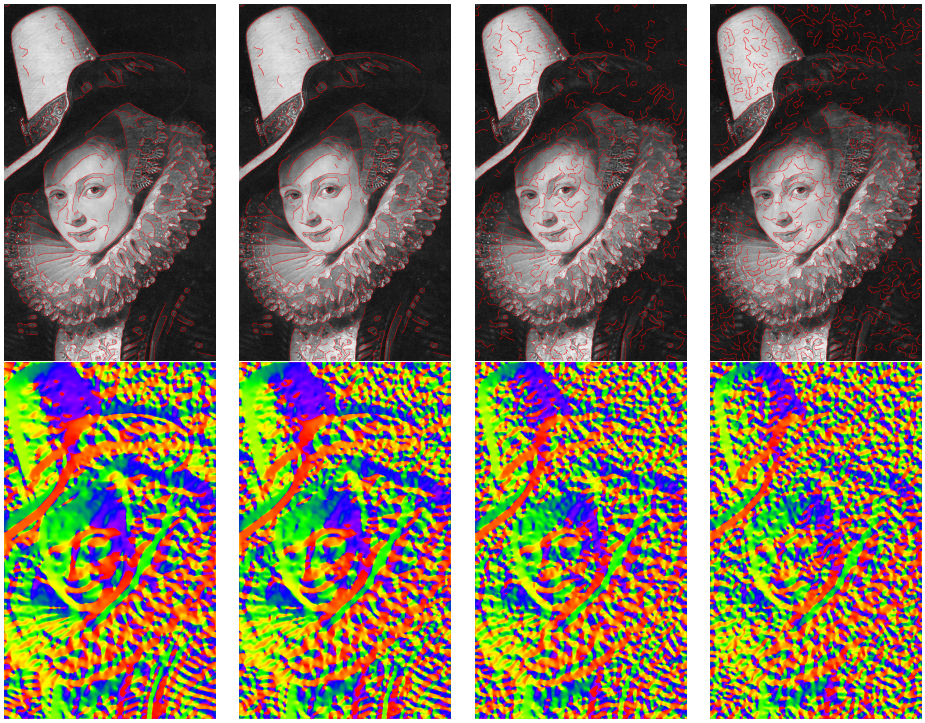
Increasing the blur introduces a trade-off between noise reduction and accurate edge detection: the stronger the blur, the less noise is present, but the more the original edges are smoothed, making them harder to detect.

We can also observe that the edges found by the algorithm become increasingly fragmented. This occurs because noise alters both the magnitude and direction of the gradient, creating many small, disconnected edges in multiple directions. By looking at Images 9a, 9b, 9c, and 9d, we can clearly see that the gradient orientation becomes more variable as noise increases. This leads to deviations in the detected edges, causing incomplete edges or false positives. We can see the same effect also reported in the 11. From the plot we can also have an idea of how much the blur effect helps in case of a noisy image reducing a lot this effect of fragmentation.

In contrast, in a noise-free image, gradient changes are smoother on flat surfaces and more pronounced along true edges.

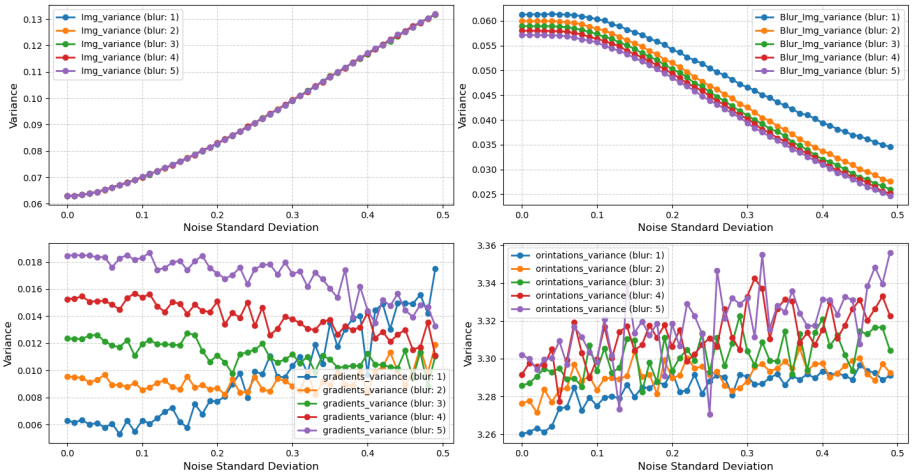
## 3.2 Question 2

Yes, the we can adjust our parameters in order to face a certain amount of noise. For what concernes the $\sigma$ parameter we can increase it to face an higher noise. Doing it, anyway, we face the problem that the original edges will be less evident since we are flattening the image. We can overcome a little bitt the problem adjusting also the thresholds. When we have noise we should be more conservative since for sure we'll have more variable gradients. So we should have to get higher thresholds. This, on the other hand can cause the missing of some true edges.

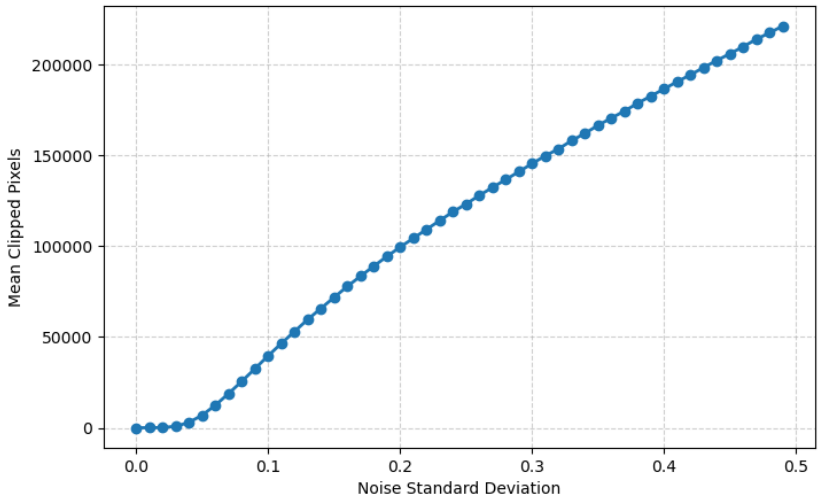


(a) $\sigma = 0.01$ (b) $\sigma = 0.1$ (c) $\sigma = 0.3$ (d) $\sigma = 0.5$

Figure 9: Comparison of results for different standard deviation of the noise.




(a) The following plots show the variances of the matrices representing the noisy image, the blurred image, the gradient magnitudes, and the gradient orientations. Regarding the descending trends, these can be explained by the fact that some values are clipped and as more noise is added, more values are clipped (see Figure 10b). This prevents us from fully observing how the overall variance changes across the different matrices. The plot of the orientation variances is particularly interesting, as it illustrates how noise affects the gradient orientations. As a consequence, we are unable to correctly detect the full edges, and the image also exhibits a fragmentation effect.




(b) Number of clipped values per std

Figure 10: Variances and # clipped values

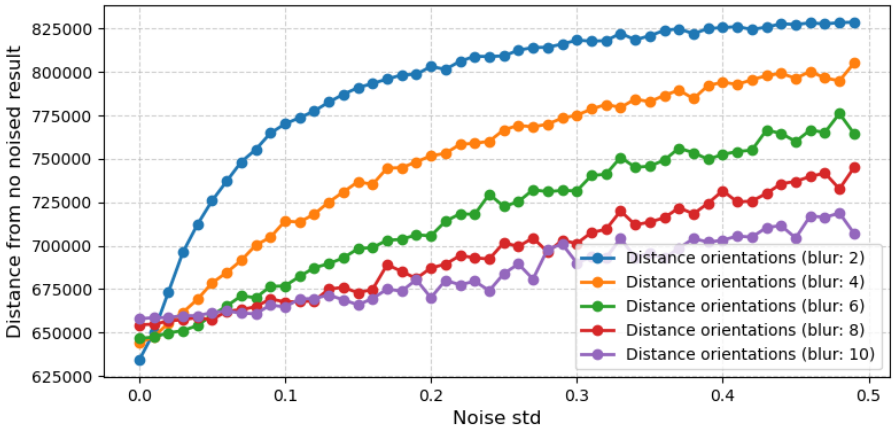



Figure 11: The plot illustrates how many gradient pixels change their orientation with respect to the noise-free image.